# 项目二：多标签非均衡样本的行人属性分类

## 一、项目目的

行人属性识别（Pedestrian Attribute Recognition, PAR）是智能监控、行人检索等应用中的基础任务：给定一张行人裁剪图，同时判断行人的性别、年龄段、朝向、衣着款式、携带物等多个属性。

通过本项目，你将：

1. 掌握图像分类任务的完整流程：数据加载与增强 → 模型构建 → 训练 → 评估 → 推理。
2. 理解**多标签分类**与**互斥多分类**的区别，以及对应的损失函数设计。
3. 理解**迁移学习**：为什么在小数据集上要使用 ImageNet 预训练权重。
4. 理解**类别不平衡**问题，并学会用指标和加权策略应对。
5. 最重要：在可运行的 baseline 之上，**亲自修改结构或损失函数**，通过对比项目验证你的想法，形成“提出假设 → 项目验证 → 分析结论”的科研闭环。

本项目鼓励使用AI编程工具。

## 二、项目任务

使用 **PA100K** 数据集训练一个行人属性识别模型，实现以下 9 个属性组、共 26 个属性的联合预测：

| 组名 | 类型 | 属性（含义） |
| --- | --- | --- |
| gender | binary | male（男性） |
| age | softmax | age_lt18 / age_18_60 / age_gt60（年龄段三分类） |
| orientation | softmax | front / back / side（朝向三分类） |
| accessory | multilabel | hat（帽子）、glasses（眼镜） |
| upper | multilabel | short_sleeve / long_sleeve / upper_stripe / upper_logo / upper_plaid / upper_splice（上装款式） |
| lower | multilabel | lower_stripe / lower_pattern / long_coat / trousers / shorts / skirt_dress（下装款式） |
| bag | multilabel | backpack / shoulder_bag / handbag（包类） |
| boots | binary | boots（靴子） |
| holding | binary | holding（正面持物） |

类型说明：

- **binary**：单个二值属性（如“是否男性”），1 维输出，用 `BCEWithLogitsLoss` 。
- **softmax**：组内属性互斥（如年龄段只能是三者之一），N 维输出，用 `CrossEntropyLoss` 。
- **multilabel**：组内属性可共存（如可以同时戴帽子和眼镜），N 维输出，每个属性独立用 `BCEWithLogitsLoss` 。

`starter_code/` 提供了一个**可直接运行的 baseline**：ResNet18 + 分组分类头 + 三种损失的朴素实现。你的任务不是从零搭建，而是**读懂 baseline、跑通 baseline、然后改进它**。

# 三、项目原理

## 3.1 图像预处理与归一化

模型输入尺寸固定为 256×128（高×宽）。预处理流程：

1. `Resize((256, 128))` ：把行人裁剪图缩放到固定尺寸（会改变宽高比，思考这对细粒度属性有何影响）。
2. `ToTensor()` ：像素从 `[0, 255]` 归一化到 `[0, 1]` ，并从 HWC 转为 CHW。
3. `Normalize(mean, std)` ：使用 ImageNet 均值/方差标准化，使输入分布与预训练模型训练时一致。

## 3.2 数据增强

训练时使用随机增强提升泛化能力。常见增强及其作用：

| 增强 | 模拟的真实变化 |
| --- | --- |
| RandomHorizontalFlip | 行人朝向镜像（注意：front/back 属性不应被翻转改变语义） |
| ColorJitter | 光照、白平衡、饱和度变化 |
| RandomErasing | 局部遮挡 |

## 3.3 卷积神经网络与迁移学习

本项目 baseline 使用 ResNet18 作为特征提取 backbone。ResNet 通过残差连接缓解深层网络的梯度消失。直接去掉其最后的 `fc` 分类层，即可获得 512 维的全局图像特征。

PA100K 仅 8 万张训练图，从零训练难以收敛到理想精度。加载 ImageNet 预训练权重后，模型已具备通用的边缘、纹理、形状感知能力，只需在行人数据上微调，收敛更快、精度更高。

## 3.4 分组多输出结构

不同属性有不同的分类语义，因此采用“共享 backbone + 每组独立分类头”的结构：

```
image -> backbone -> 512-d 特征 -> head_gender (1)
                                 -> head_age (3)
                                 -> head_orientation (3)
                                 -> head_accessory (2)
                                 -> ... 共 9 个头
```

共享 backbone 让所有属性共享底层视觉特征（如边缘、颜色），独立分类头让每组使用适合自己语义的损失函数。

## 3.5 损失函数

- **BCEWithLogitsLoss**： `L = -[y·log(σ(x)) + (1-y)·log(1-σ(x))]` ，用于 binary 与 multilabel。内部把 sigmoid 与交叉熵合并计算，数值更稳定，因此模型输出端不要手动加 sigmoid。
- **CrossEntropyLoss**： `L = -log(softmax(x)[y])` ，用于 softmax 组。

## 3.6 类别不平衡

PA100K 中部分属性正负比例悬殊（例如 `boots` 正例不足 1%）。若不加处理，模型倾向于把所有样本预测为负例也能获得很高的“准确率”，但实际毫无用处。

应对手段：

- `pos_weight` ：在 BCE 损失中对正例加权， `pos_weight = 负例数 / 正例数` 。
- Focal Loss：自动降低易分样本的权重，聚焦难分样本。
- 评价指标上改用 mA（见 3.7）。

## 3.7 评估指标

**每属性准确率（AttrAcc，baseline 已实现）**

对每个属性分别统计预测正确的比例：

```
AttrAcc_i = (TP_i + TN_i) / N
```

组内取均值得到组指标，全模型取所有组均值。

**mA（mean Accuracy）**

mA 对每个属性分别计算正例准确率和负例准确率再平均，对类别不平衡更鲁棒：

```
mA_i = 0.5 * (TP_i / (TP_i + FN_i) + TN_i / (TN_i + FP_i))
```

**实例级指标**

把每张图片视为一个多标签样本，计算整体 Precision / Recall / F1。

# 四、项目环境

- Python 3.10+
- PyTorch >= 2.0，torchvision
- Pillow、PyYAML
- GPU：4GB 显存即可（如 RTX 2060/3060 笔记本）；无独显也可运行，训练会慢数倍，建议调小 batch 或 epoch 数。

安装依赖：

```
pip install -r requirements.txt
```

# 五、数据集说明

PA100K 是行人属性识别的经典公开数据集，由室外监控摄像头采集，包含 100,000 张行人裁剪图（80,000 训练 / 10,000 验证 / 10,000 测试），每张标注 26 个属性。

数据目录结构：

```
data/pa100k/
├── train/            # 80,000 张训练图
├── val/              # 10,000 张验证图
├── test/             # 10,000 张测试图
├── train_list.txt    # 每行：相对路径<TAB>26个0/1标签
├── val_list.txt
└── test_list.txt
```

标签列与属性的对应关系见 `starter_code/dataset.py` 中的 `ATTR_ORDER`，**顺序不可改动**。

> 数据由助教统一分发。

# 六、项目内容与步骤

## 环境与数据探索

1. 按第四节装好环境。
2. 运行数据探索脚本：

```
cd starter_code
python dataset.py
```

观察输出：样本数、各属性正例率。思考哪些属性最不平衡。

3. 写一个小脚本（自己新建文件即可）随机读取 9 张训练图，画出它们的属性标签，检查数据加载是否正确。

## 跑通 baseline

1. 通读 `starter_code/` 全部代码，理解数据流： `dataset.py` 如何组织标签， `model.py` 如何分组输出， `losses.py` 如何按组计算损失， `evaluate.py` 如何按组统计准确率。
2. 启动 baseline 训练：

```
# 仅仅参考
python train.py --epochs 15 --batch-size 64
```

3. 训练结束后用推理脚本测试几张图：

```
# 仅仅参考
python infer.py --ckpt runs/xxx/best.pt --img data/pa100k/test/000009.jpg
```

4. 在项目报告中记录 baseline 的各组准确率与逐属性准确率。**这份数字是你后续所有改进的对照组，务必保存好。**

## 提出并验证你的改进（本项目的核心）

从第七节“鼓励的修改方向”中**至少选择一项**（也可以是你自己的其他想法），实现它，并用**相同的训练配置**重新训练，与 baseline 做对比项目。

要求：

- 修改必须有明确动机（为什么这样改？预期解决什么问题？）。
- 对比必须公平：除了你修改的部分，其他超参数保持一致。
- 结果无论好坏都要记录；指标下降也是正常的项目结论，分析原因即可。

## 失败分析与报告

1. 在验证集上找出 5 张以上预测错误的图片，逐张分析错误原因（遮挡？模糊？属性本身难区分？标注噪声？）。
2. 找出最容易混淆的属性对，分析原因并提出进一步改进设想。
3. 撰写项目报告。

# 七、鼓励的修改方向

修改分为三类，可自由组合。标注 ★ 的数量表示实现难度。

## 7.1 修改损失函数

### （★）pos_weight 加权

训练时加 `--pos-weight` 即可启用（`dataset.py` 的 `compute_pos_weight` 已实现）。观察 `boots`、`holding` 等稀有属性的变化。

### （★★）Focal Loss

把 binary/multilabel 组的 `BCEWithLogitsLoss` 换成 Focal Loss：

```
def focal_loss_with_logits(logits, targets, gamma=2.0, alpha=0.25):
    p = torch.sigmoid(logits)
    ce = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")
    p_t = p * targets + (1 - p) * (1 - targets)
    loss = ce * ((1 - p_t) ** gamma)
    if alpha is not None:
        alpha_t = alpha * targets + (1 - alpha) * (1 - targets)
        loss = alpha_t * loss
    return loss.mean()
```

思考：gamma 为什么能降低易分样本的权重？

### （★★★）Margin Softmax（AM-Softmax / ArcFace 风格）

把 softmax 组（age、orientation）的普通交叉熵换成加间隔的 softmax，让类内更紧凑、类间更分离。需要先改造 head：把分类头拆成“特征提取 + 分类”两段，对特征和权重都做 L2 归一化后计算余弦相似度，再对正确类别减去 margin：

```python
class AMSoftmaxHead(nn.Module):
    """替换 softmax 组的分类头。输入特征 (B, in_dim)，输出加 margin 后的 logits。"""
    def __init__(self, in_dim, num_classes, margin=0.35, scale=30.0):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(num_classes, in_dim))
        nn.init.xavier_uniform_(self.weight)
        self.margin = margin
        self.scale = scale

    def forward(self, feats, labels=None):
        w = F.normalize(self.weight, dim=1)
        x = F.normalize(feats, dim=1)
        cos = x @ w.t()                        # (B, num_classes) 余弦相似度
        if labels is not None:                 # 训练时给正确类减 margin
            one_hot = F.one_hot(labels, num_classes=cos.size(1)).float()
            cos = cos - self.margin * one_hot
        return self.scale * cos                # 仍然是 logits，接 CrossEntropyLoss
```

注意：推理阶段（labels=None）不能减 margin。改造时要想清楚 head 的输入从哪一层接出来。

## 7.2 修改网络结构

**（★）更深的 backbone**：ResNet34 / ResNet50，注意 6GB 显存下可能需要减小 batch size。

**（★★）注意力模块**：在 backbone 与 head 之间加入 SE（Squeeze-and-Excitation）模块，让模型自动学习通道重要性：

```python
class SEBlock(nn.Module):
    def __init__(self, dim, reduction=16):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(dim, dim // reduction), nn.ReLU(inplace=True),
            nn.Linear(dim // reduction, dim), nn.Sigmoid())

    def forward(self, x):
        w = self.fc(x)
        return x * w
```

**（★★）共享底层、分组顶层**：尝试让 head 的前两层共享、最后一层独立，与 baseline 的全独立头做对比，讨论参数共享对属性间相关性的利用。

## 7.3 修改数据与训练策略

- **（★）ColorJitter / RandomErasing**：加入后对比 baseline。

- **（★）标签平滑（Label Smoothing）**：softmax 组用 `label_smoothing=0.1` 的交叉熵。
- **（★★）分组采样**：按稀有属性做加权采样（`WeightedRandomSampler`）。
- 以及其他策略（例如水平翻转）等

> 提示：改动越大，越要控制变量，一次只改一个因素，否则对比项目无法归因。

# 八、提交内容

必须提交：

1. Python 源代码（requirements.txt、README 运行说明、完整的代码）；
2. 模型文件（"best.pt"）与配置文件；
3. 最终Demo（提供完整的推理代码、以及至少5涨测试图的输出截图）；
4. 项目报告；